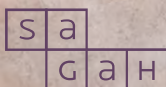


PENSAMENTO COMPUTACIONAL



SOLUÇÕES
EDUCACIONAIS
INTEGRADAS

Pensamento computacional: abstrações

Marcelo da Silva dos Santos

OBJETIVOS DE APRENDIZAGEM

- > Conceituar abstração.
- > Identificar abstrações em camada.
- > Classificar os riscos e as limitações do uso de abstrações.

Introdução

A abstração é conceito essencial do pensamento computacional, capaz de criar representações simplificadas de qualquer ser vivo ou objeto. Quando em excesso, os detalhes sobre a entidade podem desviar o foco da solução ou trazer complexidade extra para o processamento. Modelos abstratos são muito utilizados em várias áreas do conhecimento. Por exemplo, um cientista cria modelos simplificados ignorando condições climáticas, atrito e resistência do ar, e mesmo que não reflitam completamente a realidade, são capazes de simular situações específicas e resolver grandes questões.

Abstrair é uma habilidade do ser humano, que consiste na capacidade de expressar algo de maneira concisa, eliminando detalhes desnecessários. Saber trabalhar com diferentes camadas de abstração é um requisito para o profissional da computação e, ao mesmo tempo, um constante desafio. Diversos riscos e limitações a seu uso estão presentes, algumas vezes não sendo possível solucioná-los, apenas adaptar a abstração para absorvê-los.

Neste capítulo, você vai conhecer o conceito de abstração e como podemos definir diferentes níveis e camadas de abstração. Também serão apresentados os riscos e limites para o uso de abstrações.

Abstração e pensamento computacional

O **pensamento computacional** pode ser compreendido com uma estratégia para resolução de problemas utilizando conceitos computacionais (WING, 2006). A proposta é utilizar a forma de raciocinar de um cientista da computação, mas não atrelar a solução ao uso do computador. Até porque a maioria desses conceitos foram originados antes mesmo da criação do computador eletrônico, e sua aplicação transcende a ciência da computação (RILEY; HUNT, 2014). Além de estimular o pensamento lógico e sistemático, também promove o pensamento crítico, estratégico e criativo.

Como comentado anteriormente, o pensamento computacional faz uso de conceitos emprestados da computação, que dão suporte para a definição de seus fundamentos, também chamados de **pilares do pensamento computacional**: a decomposição, o reconhecimento de padrões, a abstração e os algoritmos (WING, 2006). Cada um deles trabalha em conjunto com os demais para atingir o objetivo. A **decomposição** reduz um problema em partes menores e mais simples de resolver, e a **abstração** simplifica essas partes, delas retirando os detalhes sem relevância. Já o **reconhecimento de padrões** nos auxilia a identificar similaridades entre algumas partes do problema ou comparar com problemas já resolvidos, auxiliando no reaproveitamento da experiência obtida em problemas anteriores. Ao final, temos compreensão do que deve ser feito e um planejamento das tarefas. Com base nesse planejamento, criamos listas ordenadas de ações sequenciais, os **algoritmos**, que executam o planejamento e encontram a solução buscada.

A seguir, você vai conhecer melhor o conceito de abstração.

Conceito de abstração

Segundo Wing (2006), a abstração é um conceito-chave tanto da computação quanto do pensamento computacional, chegando a descrever a ciência da computação como “a automação da abstração”. Nesse contexto, o computador é considerado uma máquina que não pensa realmente, mas interpreta o mundo a sua volta, somente executando tarefas com base na descrição de mundo realizada pelo cientista da computação.

A tarefa de descrever algo nem sempre é fácil e depende muito do ponto de vista e das experiências de quem observa. Por exemplo, imagine um automóvel qualquer. Agora o descreva utilizando, no mínimo, 300 palavras. Talvez você tenha que incluir informações sobre o motor, o desempenho, o peso, etc., para completar o texto. Agora repita a tarefa, mas utilizando somente 50 palavras. Perceba que o nível de detalhamento será diferente nos dois textos, embora o veículo seja o mesmo. Agora reflita: qual das duas descrições é melhor? Na verdade, a resposta vai depender da finalidade, de quanta informação necessitamos para resolver a tarefa.

Quando precisamos que o computador execute uma tarefa, devemos explicar somente as informações necessárias para que ele consiga executar, até porque, se não filtrarmos (e dependendo do nível de detalhes), podemos confundi-lo com o excesso de informações e desviar do objetivo principal. De maneira semelhante, o cientista cria modelos do mundo real, modelos que contêm informação suficiente para não desviar a atenção do que está analisando, e então utiliza esse modelo para definir soluções para os problemas por intermédio deles. Com esses modelos reduzidos, sem informações que desviem a atenção, quando se consegue compreender suficientemente o problema, é possível instruir o computador com algoritmos, ensinando a solução encontrada.

Esse processo de selecionar quais informações são mais ou menos relevantes é o que chamamos de **abstração**. Segundo Gutttag (2013), a essência da abstração é manter as informações relevantes de determinado contexto e omitir todo o resto. Lembrando que esse “relevante” sempre vai depender da finalidade da tarefa.

O pensamento abstrato é uma habilidade cognitiva importante do ser humano e que aplicamos a todo momento, sem perceber. Criamos abstrações, ou simplificações, para tudo que está a nosso redor, desde conceitos simples até artefatos complexos. Por exemplo, pense na palavra “número”: só de ouvir esse termo você já sabe do que se trata, mesmo sem ouvir outros detalhes. Os números são abstrações para representar a quantificação de algo.

Agora observe um exemplo mais robusto, a criação de um mapa (BEECHER, 2017). Ele é uma representação simplificada da realidade. Nem todas as informações necessitam estar lá e, dependendo de sua finalidade, também não é necessário que seja totalmente fiel à realidade. Mas qual é a utilidade de algo se não reproduz a realidade? Vejamos um exemplo: observe o mapa que representa a linha férrea de uma região do Sul do Brasil (Figura 1).

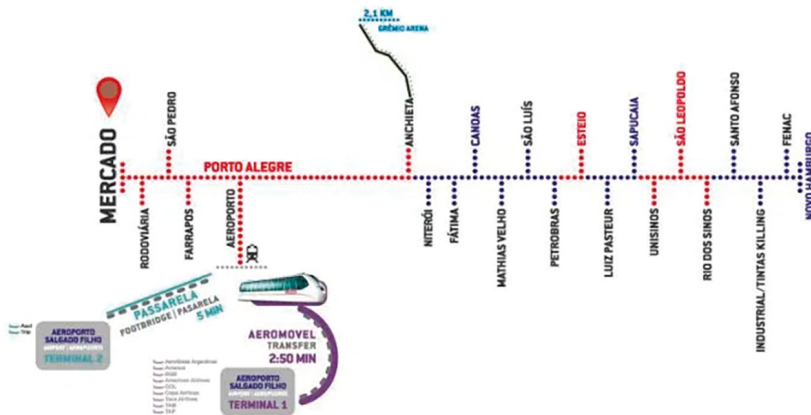


Figura 1. Representação do mapa metroviário.

Fonte: Adaptada de Beecher (2017).

Note que ela não representa a região de forma realista, começando pela disposição das paradas ao longo da linha, que não são equidistantes entre si. Considerando que o objetivo desse mapa é apresentar a ordem das estações, a omissão de detalhes acaba por simplificar sua utilização pelos passageiros. Portanto, um mapa é um exemplo bem comum de abstração, e dependendo de sua finalidade, podemos ter diferentes versões dele. Veja a Figura 2.



Figura 2. Outras representações do mesmo mapa metroviário com diferentes níveis de abstração.

Fonte: Adaptada de Beecher (2017).

Essa diferença no nível de detalhamento das abstrações está ligada, como já comentado, a sua finalidade. Aumentar ou reduzir o nível de detalhamento de um elemento é o que chamamos de camadas de abstração, assunto da próxima seção.

Abstrações em camadas

Como visto anteriormente, a abstração é a capacidade de identificar os detalhes importantes do que se está observando, mas a quantidade desses “detalhes” que atribuímos ao elemento depende do objetivo em que será empregado. O volume de informações vai impactar significativamente a representação. Também vimos que é possível criar diferentes versões de representação para o mesmo elemento, com níveis diferentes de detalhamento. Note, na Figura 3, uma sequência de estudos do pintor Pablo Picasso, com diversas representações do mesmo elemento (o touro), somente diferindo o nível de detalhes.



Figura 3. Compilado de diversos estudos do artista Pablo Picasso, apresentando diferentes níveis de abstração sobre o mesmo elemento. Os quadros estão expostos no Norton Simon Museum.

Fonte: Adaptada de Norton Simon Museum (c2020).

Essa diferença entre os níveis de abstração é conhecida como **camadas de abstração**. O conceito é simples e podemos utilizar outro exemplo do cotidiano. Dessa vez, vamos observar a utilização de areia como material de construção. Em diversos momentos, o material é utilizado e, em cada um, necessita de uma granularidade (mais “fina” ou mais “grossa”) diferente. Mas o material à disposição é muito granular. Podemos, então, utilizar peneiras para obter o material na granularidade correta. Para estruturas e vigas, não precisamos nos preocupar tanto: utilizamos uma peneira com furos maiores somente para retirar pedras e outros tipos de sujeira. Para paredes, piso e outros processos, precisamos de areia com média granularidade, pois afeta o alinhamento da estrutura. Já para o acabamento, precisamos de material bastante refinado, então nosso refinamento deve ser o maior possível, deixando passar somente os menores grãos.

Por mais estranho que possa parecer, nosso exemplo demonstra o uso de camadas (peneira), em que cada uma delas só disponibiliza a granularidade adequada para cada contexto. Cada camada representa uma quantidade diferente de informações e, cada vez que a trocamos, podemos aumentar os detalhes (reduzindo a abstração) ou adicionar mais camadas de abstração para suprimir detalhes (aumentando a abstração).



Saiba mais

Segundo Abowd e Dey (1999), **contexto** é qualquer informação que pode ser utilizada para identificar a situação de uma entidade (seja uma pessoa, lugar ou objeto) considerada relevante para a interação entre um usuário e uma aplicação. Ou seja, para uma pessoa, o contexto pode ser o lugar onde ela se encontra (em casa, no *shopping*, na faculdade), o grupo social (amigos, colegas de faculdade, parentes), etc. Informações contextuais são muito utilizadas em várias áreas da computação, como na computação ubíqua, cujo objetivo é oferecer serviços sem mesmo percebermos, baseado em perfis de comportamento criados com base nas informações contextuais do usuário.

O número de camadas ou níveis de abstração para a representação de uma entidade pode ser infinita, e a definição de qual camada é mais apropriada depende do contexto em que será empregada; ou seja, no caso da aplicação de abstrações, a escolha da camada ideal deve levar em conta o contexto. Como exemplo, podemos instruir um aplicativo a emitir alertas sobre promoções para produtos que o usuário tenha pesquisado recentemente (contexto do usuário) ou aumentar o nível de abstração e realizar as recomendações com base nas pesquisas realizadas por usuários da mesma faixa etária ou da mesma região (contexto de idade ou contexto regional).

Em outro exemplo, mais próximo ao contexto computacional, podemos comparar os níveis das linguagens de programação. Nos primeiros computadores, a programação era feita diretamente em linguagem de máquina, utilizando operadores booleanos (zeros e uns) em sua escrita. Esse processo era realizado, muitas vezes, diretamente no *hardware*. Quando passamos a substituir longas instruções em linguagem de máquina por instruções mais curtas e compreensíveis, cada vez mais próximas à linguagem natural (linguagem humana), os detalhes do *hardware* ficaram ocultos ao desenvolvedor. Ou seja, quanto mais alto o nível da linguagem, mais abstrata se torna.

Outro exemplo é quando você pressiona uma tecla qualquer do teclado do computador. Você precisa saber todos os processos (sinais, interrupções, pulsos elétricos) que estão ocorrendo naquele instante ou só necessita que o caractere seja apresentado no monitor? Caso você precise manipular o comportamento do *software* para reagir ao caractere pressionado, vai remover algumas camadas de abstração (adicionar detalhes) até que chegue a um modelo que seja adequado ao contexto.

O conceito de camadas também pode ser aplicado na organização hierárquica de entidades (seres vivos ou objetos). Agrupar um conjunto de elementos em categorias é uma forma de abstração muito comum. Vamos analisar, por exemplo, o termo “veículo”. Podemos inferir as camadas de diversas formas, e uma delas poderia ser desta forma, começando do nível mais baixo de abstração:

- no nível mais baixo, teremos Renault, Honda, Mitsubishi, etc.;
- podemos unir todos os Ford, Renault e Nissan, e assim por diante, para criar o conceito de carros, e também reunimos os Honda, Yamaha, Suzuki, entre outros, e criamos o conceito de motocicletas;
- os carros se juntam às motocicletas, minivans, etc., para se tornarem veículos.

Como foi comentado, trata-se de um exemplo resumido. Poderíamos ter iniciado em um nível mais concreto, na descrição das peças de montagem, ou poderíamos ter seguido o caminho contrário, iniciando com conceitos bastante abstratos e adicionando detalhes ao longo da análise. Por exemplo, podemos subdividir o conceito de veículos como terrestres, marítimos e aéreos.

Plataformas de vendas *on-line* utilizam esse conceito em suas aplicações. Com base nos históricos dos usuários de determinada região, são agrupados comportamentos de busca, compras efetuadas, entre outras informações.

Então, essas informações são agrupadas e é formado o perfil de consumo regional. Com base nesse perfil, quando outro usuário da mesma localidade acessa o sistema, lhe são recomendados os produtos do perfil regional, aumentando a probabilidade de venda. Sistemas de recomendação buscam produzir a melhor sugestão possível e isso pode representar reduzir o nível de abstração e levar em conta mais informações sobre o usuário. Um sistema também pode definir a divisão de perfis regionais por público (adulto, infantil, masculino, feminino, etc.).

Computacionalmente, diversas áreas empregam camadas de abstrações em seus processos, como as elencadas a seguir.

- **Modelagem de banco de dados:** modelo entidade relacionamento utiliza três camadas de abstração diferentes, um nível para cada estágio de maturidade da implementação.
- **Análise de sistemas:** em processos de análise de sistemas, são utilizados diagramas e documentos com diversos níveis de abstração, como o diagrama de caso de uso (alta abstração) da UML e o documento de descrição do caso de uso (baixa abstração).
- **Programação:** em orientação a objetos, por exemplo, entidades são representadas como classes abstratas ou interfaces (maior abstração que a própria classe abstrata), entre outras. Aqui, ainda se pode destacar o POJO (Plain Old Java Object), outro tipo de classe que utiliza o conceito de abstração para modelagem (o mais simples possível) de entidades.
- **Webdesign:** assim como o analista de sistemas, utiliza inúmeros artefatos com níveis de abstração diferente (*wireframes*, protótipos, entre outros) em estágios de maturidade diversos.
- **Hardware e redes:** na construção de *hardware* e de outros equipamentos de infraestrutura de redes, utilizam camadas e abstrações em suas representações.

Como você pode perceber, definir o nível de abstração que vai atender à necessidade e construir a melhor solução para determinado contexto podem ser tarefas bastante complexas. Também percebe que, necessariamente, não existe melhor ou pior nível de abstração: existe o mais adaptado ao problema em questão. Essa análise pode trazer alguns riscos ao processo, como você verá a seguir.

Riscos e limitações do uso de abstrações

Como visto nas seções anteriores, a abstração, além de função cognitiva inata do ser humano, é um dos pilares do pensamento computacional, permitindo que possamos representar elementos do mundo real de forma simplificada e, assim, utilizá-los na solução de problemas. Também foi apresentado como uma abstração da realidade pode ser representada de múltiplas formas, dependendo da quantidade de camadas utilizadas em sua construção.

O fato é que abstrações são conceitos muito ligados à experiência e à perspectiva do observador. Como comentado anteriormente, cada analista descreve uma abstração levando em conta as particularidades importantes para ele. Outro exercício de abstração: como você descreveria um gato? O exercício serve para recordar que não existe uma definição mais correta, mas a mais adequada para determinado cenário. Note, na Figura 4, esse exemplo da representação do gato, em que são apresentados dois pontos de vista: para a senhora idosa, o gato é uma “bola de pelos” graciosa, enquanto, do ponto de vista da cirurgia veterinária, o gato é representado por sua anatomia, esqueleto e órgãos internos. Cada uma das representações é adequada na perspectiva das observadoras.

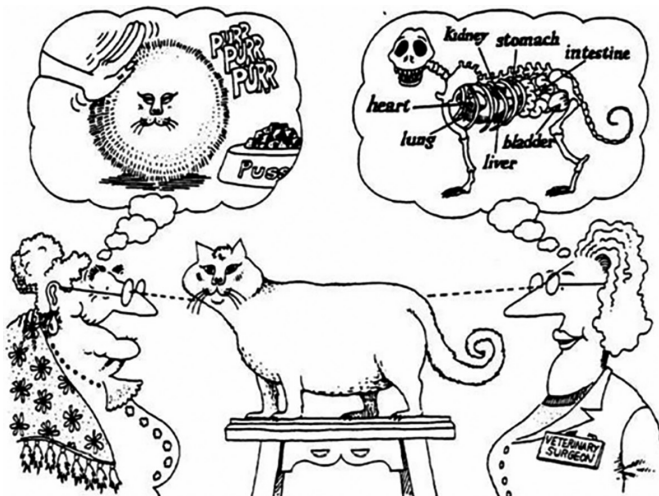


Figura 4. Experiência do analista e diferentes perspectivas podem influenciar o resultado da abstração.

Fonte: Adaptada de Booch (2004).

Abstrações são comuns em algoritmos. Linguagens que utilizam orientação a objetos a utilizam para a construção de uma classe, por exemplo, que é a abstração de um elemento do mundo real, e uma invocação de método, por sua vez, é a abstração de algum comportamento concreto.

As abstrações são essenciais para o pensamento computacional, mas é necessário atenção para que elas possam ser realmente úteis. A definição de uma abstração é uma tarefa complexa, de forma que o risco de erros está sempre presente. Um dos problemas mais comuns é o **vazamento de abstração**, que ocorre quando uma entidade vaza detalhes de sua construção, como propriedades ou comportamento específico de implementação (VAN DEURSEN; SEEMANN, 2019).

Para deixar mais claro o que é o vazamento, vamos voltar ao exemplo dos carros. Como comentamos na seção anterior, um carro possui várias abstrações, que fornecem uma interface amigável para o motorista. Pensando em sua composição (motor, eixos, sistema elétrico, sistema de frenagem, entre outros), seria complexo conhecer o funcionamento de cada uma das partes para conduzi-lo, mas, ao contrário, o carro oculta todos esses detalhes atrás interfaces mais simples, como o volante, os pedais e a alavanca de câmbio. Você não precisa saber o que ocorre internamente ao interagir com as interfaces para conduzir o veículo, e é justamente nesse ponto que vamos encontrar possíveis vazamentos. Considerando que todo motorista aprende que o pedal de embreagem deve ser pressionado somente para a troca da marcha, se não assim o fizer, haverá desgaste no sistema de freio. Essa informação sobre o funcionamento do sistema de freios nos fez descobrir um detalhe da implementação do pedal de embreagem, como é sua função e como ele afeta outros componentes da abstração.

Quando esses detalhes de implementação ficam evidentes, temos o chamado vazamento. Podemos continuar encontrando vazamentos no carro, como no câmbio manual, por exemplo: o motorista é obrigado a conhecer o mecanismo para conduzir, sendo que o carro deveria trocar as marchas sozinho. Nesse exemplo, o câmbio automático seria uma abstração sem o vazamento citado, oferecendo maior conforto e comodidade ao motorista. Embora o uso de uma abstração mais “fechada” traga alguns benefícios, o uso deve ser sempre avaliado. Por exemplo, alguns especialistas (SILVA, 2019) afirmam que o câmbio automático afeta o desempenho e a eficiência (consumo de combustível, por exemplo) devido a sua estrutura mais complexa e porque suas trocas de marchas ocorrem com rotações mais altas. A abstração mais adequada vai depender da aplicação, ou seja, de qual característica vai atender melhor o usuário: desempenho ou comodidade.

Na computação, podemos citar vários exemplos, como os *frameworks*, uma abstração com várias interfaces para a construção de sistemas. Esses pacotes auxiliam a construção, mas não resolvem todos os problemas, deixando brechas para o desenvolvedor sobrescrever suas funcionalidades: outro vazamento. No caso da função “Garbage collector” da linguagem Java, cujo papel é eliminar instâncias descartadas da memória do computador, é permitido realizar o gerenciamento manual de memória: mais um vazamento. Quando você está trabalhando com um número em representação textual e precisa convertê-lo para formato número, vazamento novamente (VAN DEURSEN; SEEMANN, 2019). Lembrando que não necessariamente tais vazamentos sejam negativos, pois permitir que um desenvolvedor experiente customize funcionalidades ou trate manualmente o gerenciamento de memória pode trazer incrementos importantes no desempenho ou no consumo do produto.

Quando vazamentos ocorrem, detalhes que deveriam ser ocultos tornam-se indispensáveis para seu funcionamento. Dificilmente uma abstração de alta complexidade será à prova de vazamentos, e raramente será possível prever o fato. Devemos, assim, aprender a tratá-los. Na medida do possível, o modelo deve ser corrigido para considerar o problema e os detalhes que se revelaram importantes, que devem ser acrescentados ao modelo de forma explícita. Por exemplo, no caso antes citado sobre o sistema de freios, o motorista acaba aprendendo detalhes da implementação para saber o momento e a forma certa de executar a frenagem. O freio ABS (*anti-lock braking system*) e seu sistema de controle de frenagem retira um pouco dessa responsabilidade do motorista, sendo uma forma de reduzir o vazamento de abstração sobre o sistema de frenagem.

Por outro lado, tentar evitar todos os vazamentos pode levar a outros problemas de implementação. Para que a abstração seja utilizada, é necessário transformar em algo concreto, ou seja, criar uma instância com detalhes concretos. Anteriormente, comentamos sobre orientação a objetos e como uma classe representa a abstração de um elemento; porém, para utilizá-la, precisamos instanciar a classe, transformá-la em um objeto com informações concretas (valores para os atributos, por exemplo) para que ela possa executar sua função. Tentar evitar todos os vazamentos pode causar falhas no detalhamento e aumentar a complexidade da codificação da abstração ou até mesmo seu formato de utilização.

Esses são riscos intrínsecos ao processo de abstração. Assim, deve-se identificar a quantidade correta de detalhamento para que o modelo fique equilibrado, sem detalhes em excesso, de forma que o modelo se torne mais complexo que o necessário, nem detalhes de menos, de modo que não tenha

os requisitos mínimos para executar sua tarefa. Também devemos evitar os vazamentos de abstração, mas sem aumentar sua complexidade sem razão. Tudo vai depender do contexto para o qual a abstração é necessária, talvez altamente abstrato para alguns casos e flexível em outros.

Abstrações podem simplificar as tarefas do desenvolvedor, facilitando seu trabalho e aumentando sua produtividade. Mas abstrações tendem a ter um impacto negativo no desempenho da aplicação. É muito difícil generalizar e ser eficiente. A criação de abstrações de *software* está longe de ser perfeita, mas, sem ela, com certeza seria mais complexo construirmos soluções de *software* computacionalmente.

Referências

ABOWD, G. D.; DEY, A. K. Towards a better understanding of context and context-awareness. In: INTERNATIONAL SYMPOSIUM ON HANDHELD AND UBIQUITOUS COMPUTING, 1999, Heidelberg. *Anais [...]*. Heidelberg: Springer-Verlag, 1999.

BEECHER, K. *Computational thinking a beginner's guide to problem-solving and programming*. Swindon: BCS, 2017.

BOOCH, G. et al. *Object-oriented analysis and design with applications*. 3. ed. Boston: Addison-Wesley, 2004.

GUTTAG, J. V. *Introduction to computation and programming using Python*. Cambridge: MIT Press, 2013.

NORTON SIMON MUSEUM. *The Bull*. c2020. Disponível em: <https://www.nortonsimon.org/art/detail/M.1977.08.3.11a,b.G>. Acesso em: 3 jan. 2021.

RILEY, D.; HUNT, K. A. *Computational thinking for the modern problem solver*. Boca Raton: CRC Press, 2014.

SILVA, J. E. da. *Câmbio automático consome mais combustível?* 2019. Disponível em: <https://www.seguroauto.org/cambio-automatico-consome-mais-combustivel>. Acesso em: 3 jan. 2021.

VAN DEURSEN, S.; SEEMANN, M. *Dependency injection principles, practices and patterns*. Shelter Island: Manning Publications, 2019.

WING, J. M. Computational thinking. *Communications of the ACM*, v. 49, nº 3, p. 33–35, 2006. Disponível em: <http://www.cs.cmu.edu/~15110-s13/Wing06-ct.pdf>. Acesso em: 3 jan. 2021.

Leitura recomendada

DENNING, P. J.; TEDRE, M. *Computational thinking*. Cambridge: MIT Press, 2019.



Fique atento

Os *links* para *sites* da *web* fornecidos neste capítulo foram todos testados, e seu funcionamento foi comprovado no momento da publicação do material. No entanto, a rede é extremamente dinâmica; suas páginas estão constantemente mudando de local e conteúdo. Assim, os editores declaram não ter qualquer responsabilidade sobre qualidade, precisão ou integralidade das informações referidas em tais *links*.

Conteúdo:



SOLUÇÕES
EDUCACIONAIS
INTEGRADAS